Sila Files API — Sending Files from Mobile

Oct 23, 2026 · For the mobile app developers · Companion to the *Sila API — Mobile Developer Guide*

Overview

Files no longer travel inside a chat call. Sending a file (or a voice note) is two calls:

- **Upload** the file to the Files API on **:7500**. It checks the type and content, scans it for viruses, stores it and returns an `attachmentID`.
- **Post** a message that points at that `attachmentID`, through `/api/v2/exec` on **:7400**, like any other call.

The file only appears in the conversation after step 2. An uploaded file that is never posted stays invisible to everyone. Opening a file also goes through :7500, with the user's token.

Item	Value
Files API	`https://sila.glcpaints.com:7500`
Operations API	`https://sila.glcpaints.com:7400` (unchanged)
Database key	`SILA101`
Auth	The same access token you get from `/api/v2/auth/login` on :7400: `Authorization: Bearer <accessToken>`. There is no separate sign-in for :7500.

Step 1 — Upload the file

`POST https://sila.glcpaints.com:7500/api/v2/attachments` as `multipart/form-data`, one file per request.

Form field	Value
`database`	`SILA101`
`entityType`	`Chat`, `Group` or `Ticket` (case matters)
`entityKey`	The conversation: the chat's `SessionID`, the `GroupID` or the `TicketID`, as text
`description`	Required, up to 200 characters. Send the file name (the web app does).
`file`	The file itself, with its real file name and extension (e.g. `invoice.pdf`, `voice-20261023-1412.m4a`).

A successful upload returns 200:

```
{ "attachmentID": 1840231, "fileID": 1840231 }
```

Keep the `attachmentID` for step 2. It can be posted once, only by the person who uploaded it, and only to the same conversation.

What the server accepts

Rule	Detail
Types	PDF, PNG, JPG/JPEG, Word (.docx), Excel (.xlsx), PowerPoint (.pptx), TXT, CSV, and audio: .m4a, .aac, .mp3, .ogg, .opus, .wav, .webm. Anything else returns 400.
Size	Up to 50 MB per file. Larger returns 413.
Content check	The bytes must match the extension. A renamed file fails. Office files must be macro-free (.docm / .xlsm renamed to .docx / .xlsx fail). TXT/CSV must be text, not binary. Audio must start with its format's header (e.g. an .m4a file is an MP4 container).
Virus scan	Every file is scanned before it is stored. An infected file returns 400 "File rejected by virus scan". If the scanner is down, the upload returns 503: ask the user to try later.
Who may upload	Chat: either person in it. Group: members; when the chat is locked, admins only. Ticket: members who haven't left, while the ticket is New or In Progress.
Content-Type	The type you send is ignored; the server decides it from the extension.

Voice notes

- Record as **.m4a** (AAC in an MP4 container), as the app does today; it plays on Android, iOS and the web. The other audio types above are accepted too.
- Upload and post a voice note exactly like any other file (steps 1 and 2); there is no separate operation.
- The message's file has an audio extension (`FileExtension` `m4a`, ...): show it as a voice note. To play it, download it as in *Opening a file* below; it is served as `audio/mp4` (`audio/mpeg`, `audio/ogg` ... for the others).

Step 2 — Post the file as a message

`POST https://sila.glcpaints.com:7400/api/v2/exec`, with the same body shape as every other call. These operations are on the web procedures and are open to the app; `extraParams` stays empty.

Type	proc	operation	lineData
Chat	`dbo.APISilaWebOperation`	`Send chat file`	`{SessionID, AttachmentID, LocalChatID, ReplayMsgID?}`
Group	`dbo.APISilaWebOperation`	`Send group chat file`	`{GroupID, AttachmentID, LocalChatID, ReplayMsgID?}`
Ticket	`CHA.APISilaWebTicketOperation`	`Send ticket file`	`{TicketID, AttachmentID, LocalChatID, AdminOnly?}`

Example (ticket):

```
POST https://sila.glcpaints.com:7400/api/v2/exec
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "database": "SILA101",
  "proc": "CHA.APISilaWebTicketOperation",
  "operation": "Send ticket file",
  "lineData": { "TicketID": 2301855, "AttachmentID": 1840231, "LocalChatID": 917 },
  "extraParams": {}
}
```

- `LocalChatID`: required, a number the app picks, unique per message for this user (as in `New Chat`).
- `ReplayMsgID`: optional, the `ChatID` being replied to.
- `AdminOnly`: tickets only; `1` posts the file as an admins-only note (ticket admins and team leaders only).
- The reply's `data.List0[0]` is the saved message, with its `ChatID`, `AttachmentID`, file name and size.

The file then shows in the existing message lists (`Get Right Side Ticket`, `Get Left-Right Side One-to-one`, `Get Left-Right Side Group`) like any other file, with `AttachmentID`, `FileOrginalName`, `FileSize` and `FileURL`. The other members get the usual live update.

Opening a file

A file uploaded this way has a `FileURL` containing `/api/v2/files/`. **Don't open that link from the app**: it works only inside the office network. Download it through the Files API with the token instead:

```
GET https://sila.glcpaints.com:7500/api/v2/attachments/{AttachmentID}/download
    ?database=SILA101&entityType=Ticket&entityKey=2301855&inline=true
Authorization: Bearer <accessToken>
```

- `inline=true`: PDFs, images and audio come back for viewing / playing; other types always download.
- `preview=true`: for images, a small JPEG (at most 480 px) for the chat bubble. Load the full file only when the user opens it.
- Voice notes: download once when the user presses play, keep the file for the session, and play it locally (the player can't add the Authorization header to a streaming URL on every platform).
- `entityType` / `entityKey` must be the conversation the file was posted in; the server checks that the user is still in it.
- Older files whose `FileURL` does **not** contain `/api/v2/files/` (sent before this change) keep opening from their own link, as today.

Code example (Flutter, package:http)

```
import 'dart:convert';
import 'package:http/http.dart' as http;

const filesApi = 'https://sila.glcpaints.com:7500';
const api = 'https://sila.glcpaints.com:7400';

/// Step 1: returns the AttachmentID. Throws with the server's message on failure.
Future<int> uploadFile(String token, String entityType, int entityKey,
    String path, String fileName) async {
  final req = http.MultipartRequest('POST', Uri.parse('$filesApi/api/v2/attachments'))
    ..headers['Authorization'] = 'Bearer $token'
    ..fields['database'] = 'SILA101'
    ..fields['entityType'] = entityType           // Chat | Group | Ticket
    ..fields['entityKey'] = '$entityKey'          // SessionID | GroupID | TicketID
    ..fields['description'] = fileName.length > 200 ? fileName.substring(0, 200) : fileName
    ..files.add(await http.MultipartFile.fromPath('file', path, filename: fileName));
  final res = await http.Response.fromStream(await req.send());
  final body = res.body.isEmpty ? {} : jsonDecode(res.body);
  if (res.statusCode != 200) throw Exception(body['message'] ?? 'Upload failed (${res.statusCode})');
  return body['attachmentID'] as int;
}

/// Step 2 (ticket): posts the file; returns the saved message row.
Future<Map<String, dynamic>> sendTicketFile(
    String token, int ticketId, int attachmentId, int localChatId) async {
  final res = await http.post(Uri.parse('$api/api/v2/exec'),
      headers: {'Authorization': 'Bearer $token', 'Content-Type': 'application/json'},
      body: jsonEncode({
        'database': 'SILA101',
        'proc': 'CHA.APISilaWebTicketOperation',
        'operation': 'Send ticket file',
        'lineData': {'TicketID': ticketId, 'AttachmentID': attachmentId, 'LocalChatID': localChatId},
        'extraParams': {},
      }));
  final body = jsonDecode(res.body);
  if (res.statusCode != 200) throw Exception(body['message']);
  return body['data']['List0'][0];
}

// A voice note: the same two calls, e.g.
//   final id = await uploadFile(token, 'Ticket', ticketId, recordedPath, 'voice-$stamp.m4a');
//   await sendTicketFile(token, ticketId, id, localChatId);
```

Show upload progress by streaming the file (e.g. with `dio`'s `onSendProgress`); the server needs nothing different.

Errors

Every error reply is JSON with a `message` to show the user. Upload and download errors come from :7500; a refused post comes back from :7400 as 422, like any other operation.

Status	From	Meaning	What the app does
400	Files API	Type not allowed, content doesn't match, empty file, missing field, or rejected by the virus scan	Show the message; don't retry.
401	Both	Token missing or expired	Refresh once and retry.
403	Files API	Not allowed to send here (locked group, closed ticket)	Show the message.
404	Files API	Conversation or file not found, or the user is no longer in it	Show the message.

Status	From	Meaning	What the app does
413	Files API	File over 50 MB	Tell the user the limit.
422	Post	The procedure refused: the ticket is not New / In Progress, LocalChatID missing, or "That file can't be sent here" (the AttachmentID belongs to another conversation or user, or was already posted)	Show the message. For "can't be sent here", upload the file again.
503	Files API	Virus scanner unavailable	Ask the user to try again later.

Retries and big files

- Before a large upload, refresh the token if it expires within a minute: access tokens last 5 minutes.
- If an upload's reply is lost, upload again. The first copy is never posted, so nobody sees it.
- If the post's reply is lost, resend it with the **same** `LocalChatID` and `AttachmentID`: the server returns the message it already saved instead of posting it twice.
- Upload several files one after another, or two at a time at most, then post each one.

Checklist

- Upload to :7500 with the :7400 access token; one file per request; real file name and extension.
- Check the type and the 50 MB limit in the app before uploading, for a quicker message.
- Voice notes: record .m4a; upload and post like any file.
- Post with the operation for the conversation type and a unique LocalChatID.
- Open new files through /attachments/{id}/download, never through their FileURL.
- Use preview=true for image thumbnails.